

FPVR request API workflow

DCSA API on Databricks Apps. Callers authenticate with client id and secret, open an official request for one FPVR report, poll until ready, then fetch the payload. Request logging uses the app service principal; report data is queried on-behalf-of the caller so Unity Catalog ABAC and RBAC apply.

4 Caller steps	3 Public APIs	5 Source tables	7 FPVR reports in the mock
--------------------------	-------------------------	---------------------------	--------------------------------------

The mock covers all seven FPVR reports, including FPVR-6 Human Capital, using the five edladmin streaming tables.

Caller flow

Machine-to-machine. Token is OAuth2 client credentials, then three HTTP calls. Entity resolution happens inside the request API, before an official request id is issued.

STEP	CALL	SUCCESS	OTHERWISE
1. Auth	POST /oauth/token (client_id + client_secret)	Bearer token	401 invalid credentials
2. Request	POST /api/v1/requests	201 + request_id (unique subject)	409 candidates without SSN, or 404 none
3. Status	GET /api/v1/requests/{id}	pending ready failed	403/404 if not the caller's request
4. Response	GET /api/v1/requests/{id}/response	Report JSON (OBO query)	409 if not ready; empty/denied if ABAC hides rows

Entity resolution before an official request

No subject 0 matches HTTP 404. Nothing is written to the request log. Caller must tighten or correct identity fields.	Ambiguous 2+ matches HTTP 409. Return candidate list without SSN (name, DOB, place of birth, sex, citizenship) plus an opaque candidate_id. No request id yet. Caller resubmits with candidate_id or more filters.	Official request 1 match HTTP 201. App SP inserts a request row, generates request_id, stores report code, caller identity, timestamps. Status starts pending or ready depending on whether the report is in-lake.
--	---	---

Auth and identity split

Token (caller)

OAuth2 client_credentials against the Databricks workspace (service principal client id and secret). Use that Bearer on every API call. Do not mint a custom token store.

The same token is the OBO credential for SQL Statement Execution when serving the report.

Who runs which SQL

App SP: INSERT/UPDATE on the request log table only (audit must always land).

Caller OBO: SELECT on subject_* tables when assembling a ready response, so row filters, column masks, and grants apply.

Request log (App SP)

Table: dcsa_catalog.dcsa_api.api_request_log

COLUMN	PURPOSE
request_id	Public handle returned to the caller
report_code	FPVR-1 ... FPVR-7
subject_ssn	Internal join key; never returned on status or candidates
status	pending ready failed
requested_by	SP client id or forwarded user
requested_at	Official request time
ready_at	When payload became servable
resolution_method	unique_match candidate_id

Report mapping (5 tables, 7 reports)

REPORT	NAME	EDLADMIN SOURCES	TYPICAL READINESS
FPVR-1	Application Bundle	identity + status + check	Ready when identity is resolved; available sections may be empty
FPVR-2	Investigation Summary	check + activity	May stay pending until checks land
FPVR-3	Adjudication Summary	status + check	Ready when status exists
FPVR-4	CV Summary	identity + status	Usually ready in-lake
FPVR-5	CV Alerts	signal	Ready with zero or more alerts

REPORT	NAME	EDLADMIN SOURCES	TYPICAL READINESS
FPVR-6	Human Capital	identity + status	Ready when identity is present
FPVR-7	Favorable to Onboard	status	Ready if STATUS_CODE is present

Source: dcsa_catalog.edladmin streaming tables · subject_identity, subject_status, subject_check, subject_activity, subject_signal.

Proposed modules

models

RequestCreate, Candidate, RequestAccepted, RequestStatus, Report payload schemas per FPVR code.

services

EntityResolutionService (OBO identity lookup), RequestLogService (App SP), ReportAssemblerService (OBO).

routes

POST /requests, GET /requests/{id}, GET /requests/{id}/response. Keep existing /health and /me. Token via POST /oauth/token.

Pending reports in this mock can flip to ready on a status poll by re-checking the lake (no external fetch yet). That keeps the poll contract without a job queue. A later worker can update status asynchronously using the same request_id.